CMPS 350 Web Development Course Project Phase 1
Social Media Platform

Weight of the project (phase 1):

- 10% of the course grade.

Important Dates

- The project phase 1 submission is due by **23:59 Thursday 12 March 2026**.
- No late submission is allowed

Description of the project:

You will build a social media platform like Twitter, Instagram, TikTok etc. This document outlines the requirements for the first phase of the social media platform. The goal is to build a user-friendly and visually appealing interface that is both **functional** and **responsive**.

- There is a single user type: **Users** who post messages and follow others, and do other activities.
  - Each user has a user *id (unique for each user)*, user name (shown in their profile), email address, password, profile picture.

Core Features

These are the essential features that the front-end must support.

Feature	**Description**
User Authentication	Users should be able to create an account and log in. This includes a registration form with validation (e.g., for email format and password strength) and a login form.
User Profile	A dedicated page for each user, displaying their information (e.g., username, bio) and their posts. The logged-in user should be able to edit their own profile.
News Feed	The main page of the platform, displaying a stream of posts from the users they follow. Each post should show the author's username, the content of the post, and the timestamp.

Create/Delete and View Posts	Users must be able to create new posts (text-based to start) or delete them. They should also be able to view a single post in more detail, along with its comments.
Like and Comment on Posts	Users should be able to "like" a post, and the like count should update. They should also be able to add comments to a post.
Follow/Unfollow Users	Users should be able to follow and unfollow other users to see their posts in the news feed.

Technical Stack & Constraints

This is the required technology stack. No external libraries or frameworks are permitted.

Technology	Requirement
HTML5	Use semantic HTML5 to give the page proper structure and meaning (<nav>, <main>, <section>, etc.).
CSS3	Use Flexbox and Grid to create all layouts. The application must be responsive and adapt to different screen sizes purely with CSS. No CSS frameworks like Bootstrap are allowed.
Vanilla JavaScript (ES6+)	All interactivity must be built with plain JavaScript. This includes: - DOM Manipulation (creating/updating/deleting elements) - Event Handling (listening for clicks, form submissions) - Managing all application data locally.
Git & GitHub	The entire project must be managed with Git for version control and hosted on a public or private GitHub repository.

Data Handling

Since there is no back-end, all data must be stored and managed within the browser.

Requirement	Description

Local Storage	Use the browser's localStorage API to make the data persist between page reloads and browser sessions.
JSON Format	All data (user accounts, posts, likes, comments) should be structured and stored in localStorage as JSON. You will need to use JSON.stringify() to save the data and JSON.parse() to retrieve it.
Data Structure	Plan a clear structure for your JSON data from the start. For example, you might have top-level arrays for users and posts.

2. Deliverables and important notes

This is a group project, and your team is responsible for making all of its own design and implementation choices. During the weekly office hours, your group can present and discuss your design and progress with the instructor. You should view the instructor as the client who is asking for the platform; they will provide high-level advice and feedback, but your team plays the role of the application designers and implementers. The final technical and design decisions are your group's responsibility to make collaboratively.

What to submit:

You are required to submit source

- code of your project,
- the URI of your GitHub repository,
- a 2-3 minute video that shows your system. You can take a recording of your screen and show how your system works (Not the code, but actual running system)
- a project report. Your final report is a document that serves two main purposes: it explains what your group has built and guides the instructor on how to use and evaluate your social media platform.
- A self-evaluation report where each group member evaluates themselves and other group members. (A template will be provided for this document)

Keep your reporrt clear, concise, and well-organized. Here is the required structure for your report:

1. Title Page

Include the following details:

- Project Title: The name of your social media platform
- Course Name: Web Development Fundamentals
- Group Members: List the full names of all members.

---

2. Project Overview

In a brief paragraph, summarize your project. Mention that you have built a front-end for a social media platform using only HTML, CSS (Flexbox/Grid), and Vanilla JavaScript, with data stored locally in the browser's localStorage.

---

3. Features Checklist

This section details what you accomplished. List the features you implemented and mark their status. This helps the instructor quickly see your progress.

Example:

Feature	Status	Notes (Optional)
User Registration & Login	☑ Implemented	Users are stored in localStorage.
News Feed	☑ Implemented	Displays all posts chronologically.
Create New Post	☑ Implemented	New posts are added to the top of the feed without a page reload.

Like a Post	☑ Implemented	Like count updates and persists.
Comment on a Post	☑ Implemented	Comments are saved and displayed under each post.
User Profile Page	☑ Implemented	
Edit User Bio	❌ Not Implemented	Ran out of time.

4. How to Use Your Application (A Quick Guide)

This is the most important part of the report. Provide a simple, step-by-step guide on how to test the main functionalities of your website.

Example:

1. Register a New User:
    - Navigate to the Register page.
    - Enter a username and a simple password, then click "Register".
    - You will be redirected to the Login page.
2. Log In and Create a Post:
    - On the Login page, enter the credentials you just created.
    - You will be taken to the News Feed.
    - Use the "What's on your mind?" text box at the top to write a post and click "Post". Your new post will appear at the top of the feed.
3. Interact with Posts:

- Click the "Like" button on any post to see the count increase.
- Type a comment in the comment box under a post and click "Comment" to add it.

---

5. Project Links

Provide the following essential links at the very end of your report:

- GitHub Repository URI: The full link to your group's GitHub repository.
- Live Demo URI (Optional but Recommended): If you have deployed your project (e.g., using GitHub Pages), include the link to the live website.

3. Grading rubric

Criteria	Excellent – 100%	Good – 80%	Satisfactory – 60%	Needs Improvement – <40%
UI & Design (35%)	The UI is exceptionally clean, responsive, and intuitive. Advanced and effective use of Flexbox and Grid is evident. The design is consistent and visually appealing across all components and screen sizes.	The UI is clear and mostly responsive. Good use of Flexbox and Grid is demonstrated. There may be minor design inconsistencies or responsiveness issues on some screen sizes.	The UI is functional but basic. Layouts work on desktop but have significant responsiveness issues. The use of Flexbox/Grid is minimal or not fully correct. The design is inconsistent.	The UI is confusing, broken, or not responsive. There is little to no evidence of proper layout management with Flexbox/Grid. The application is difficult to use due to poor design.
Functionality (35%)	All core features are fully implemented and work flawlessly. Vanilla JS is clean, well-commented, and efficiently handles DOM manipulation and events. Data persistence in localStorage is robust and error-free.	Most core features are implemented and functional, though some may have minor bugs. The JavaScript code is functional but could be better organized. Data persistence works for the most part but may have edge cases where it fails.	Some core features are incomplete or have significant bugs. The JavaScript code is messy, hard to follow, or inefficient. Data does not persist correctly across sessions.	Major features are missing or broken, making the application unusable. The JavaScript code has critical errors, and core functionality does not work as required.
Project Report (10%)	The report is exceptionally clear, professional, and complete. The "How to Use" guide is logical and allows for easy testing of all	The report is clear and contains all required sections. The "How to Use" guide is understandable, but	The report is missing one or more required sections or the information provided is unclear. The "How to Use" guide is	The report is incomplete, poorly written, and lacks critical information. Key

	features. All sections are well-detailed and provide a full picture of the project.	may miss a few steps. Some sections could use more detail.	confusing or incomplete, making it difficult to test the application.	sections like the user guide or features list are missing.
Demo Recording (10%)	The video recording shows all functionalities are working in a very clear way.	The recording is clear but a few functionalities are not demonstrated.	The video is missing misses 3-4 features and/or fails to show them in a clear way.	The recording misses many features, the explanations are not clear.
Self Evaluation Report (5%)	The students provide a well-thought self-evaluation report with great detail. The scores for students and their explanation are consistent.	The evaluation report provide a well-thought document but some details are missing	The report provides a brief information about user activities.	The report is not submitted or does not provide any useful information.
GitHub Repo (5%)	The GitHub repository is provided and well-organized. The commit history is clean, descriptive, and clearly shows collaborative work from all group members throughout the project's development.	The GitHub repository is provided and contains the final code. However, the commit history may be messy, lack descriptions, or show that work was not evenly distributed or collaborative.	A link to the GitHub repository is provided, but it is disorganized, the code is not up-to-date with the final submission, or it is empty.	No GitHub repository link is provided, the link is broken, or the repository is private/inaccessible.